Reinforcement Learning

1. Robots → walk
2. self driving cars
3. Go, Chess games

→ trial & error → win

maximize rewards

Agent → Environment

Rewards

Reinforcement Learning

Autonomous

1. 2013 DeepMind → Deep Q-Network

RL + DL
↓
integrate

2. 2016 Go
↓
AlphaGo → Lee Sedol

Reinforcement Learning

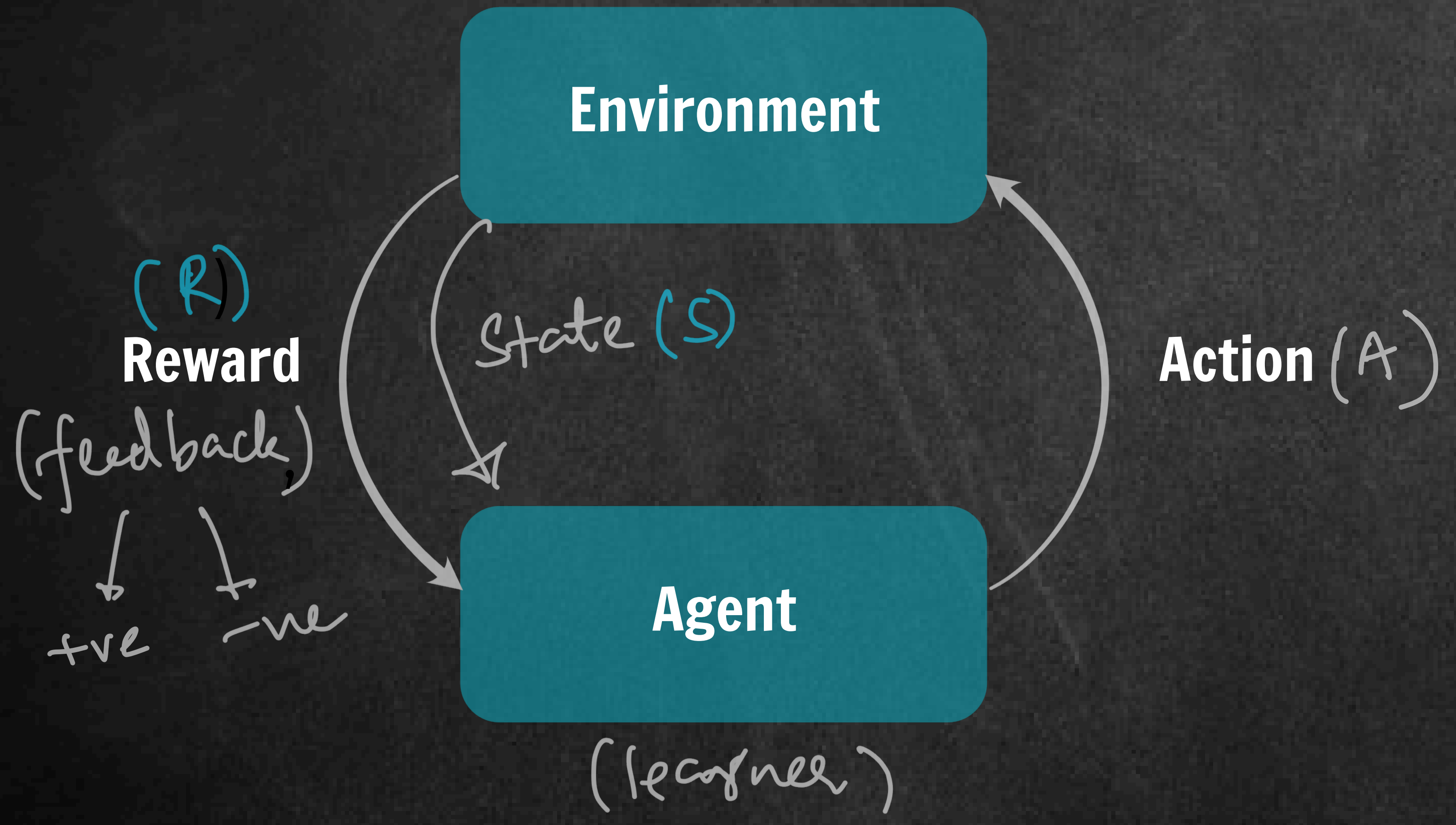
Components

S → snapshot of env

1 S_t, 2 A_t → 3 R_{t+1}

4 Policy (π) → strategy
choosing our Action

5 (G) Return (Total Future Rewards)
→ Gain



↓

ultimate goal

⇓

maximize Rewards

Reinforcement Learning

Components

Reinforcement Learning

Components

$\gamma = 0$ immediate returns

$\gamma = 1$ long term

1. Action (A_t)
2. Rewards (R_t)
3. State (S_t)
4. Policy (π) $S \rightarrow A$ $\pi\left(\frac{a}{s}\right)$
5. Returns (G)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

$\gamma \rightarrow$ discount factor

$$\boxed{0 \le \gamma \le 1}$$

Reinforcement Learning

Components

(6) Value function (VF)

↓

Returns estimate based on a particular state s.

(a) State VF → $V_\pi(s)$ estimate return based on state s followed by policy π

(b) Action VF → $Q_\pi(a,s)$

- optimal State VF → $V_*(s)$ → max. possible returns of all policies

- optimal Action VF → $Q_*(a,s)$

Reinforcement Learning

Markov DecisionProcess(MDP) — math model

sequence of State, Action, Reward . . . . . .

$[S_0, A_0, R_1, \underline{S_1}, A_1, R_2 \ldots]$

price

move

$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3 \ldots \ldots$

continuous

stock trading algos

terminate

chess / Go

Episode

Reinforcement Learning

Markov DecisionProcess(MDP)

MARKOV Property - rule

idea: next state ← curr state, curr action
not the past history

$$S_{t+1} \leftarrow S_t, A_t$$

limitations

(1) not all situation/env fall under MDP

(2) Reward → single val

Reinforcement Learning

Markov DecisionProcess(MDP)

State (S)

agent

Action (A)

Policy $P\left(\frac{a}{s}\right)$

goal → maximize the total future rewards

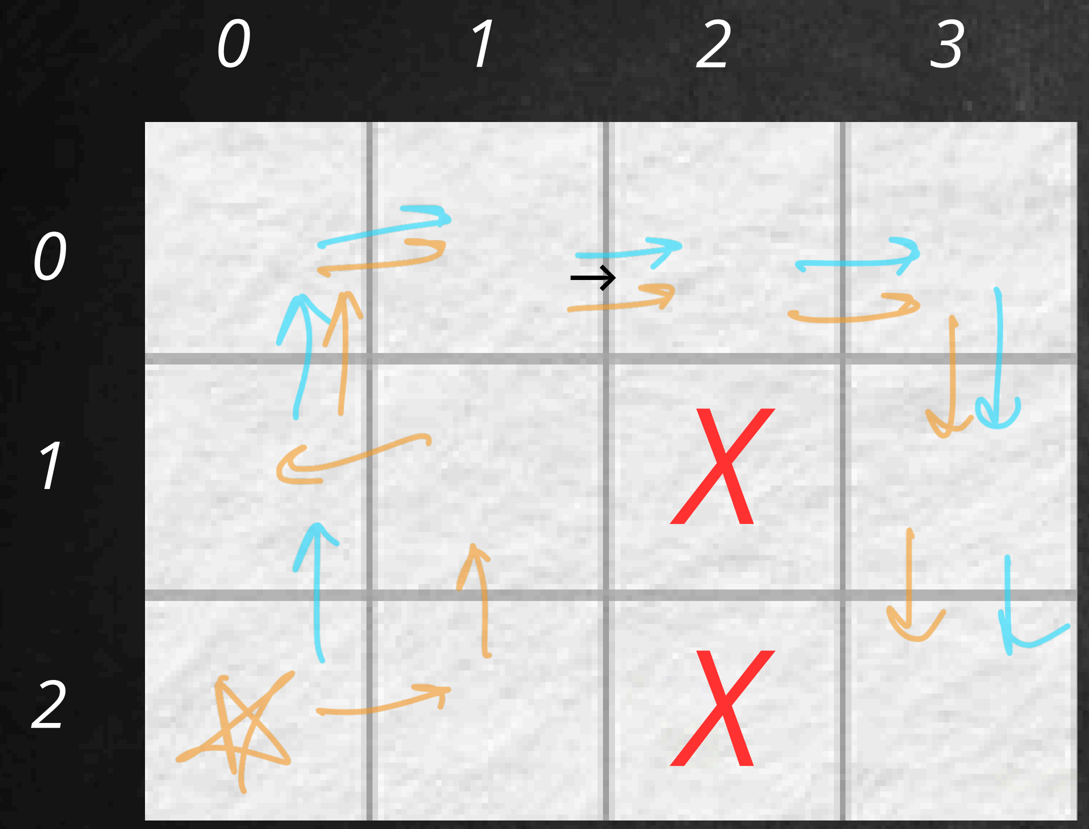
Returns

1. Q-Learning
2. SARSA
3. Policy Gradients

Reinforcement Learning

Grid Example

APNA COLLEGE



allowed moves ⇒ up, down, left, right
0 1 2 3

terminate ⇒ E reach OR 15 moves moves

MDP

S = cell (i, j) → 2 nums

A = 0-3 → 4 nums

R = target (E) → 0
non-E cells → -1

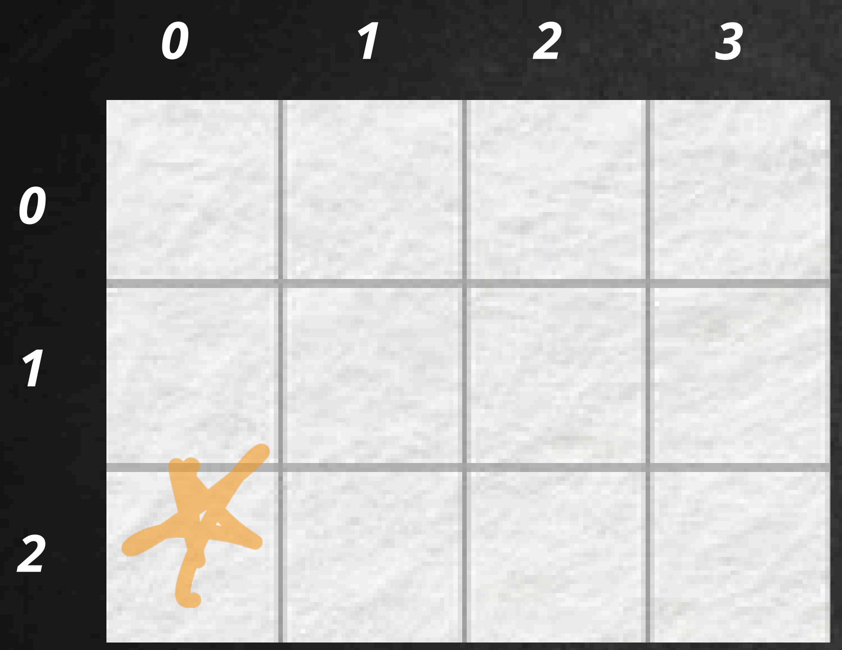
Reinforcement Learning

Grid Example

Policy optimization

World Model → understand how the env behaves

1. Model based RL
2. Model free RL



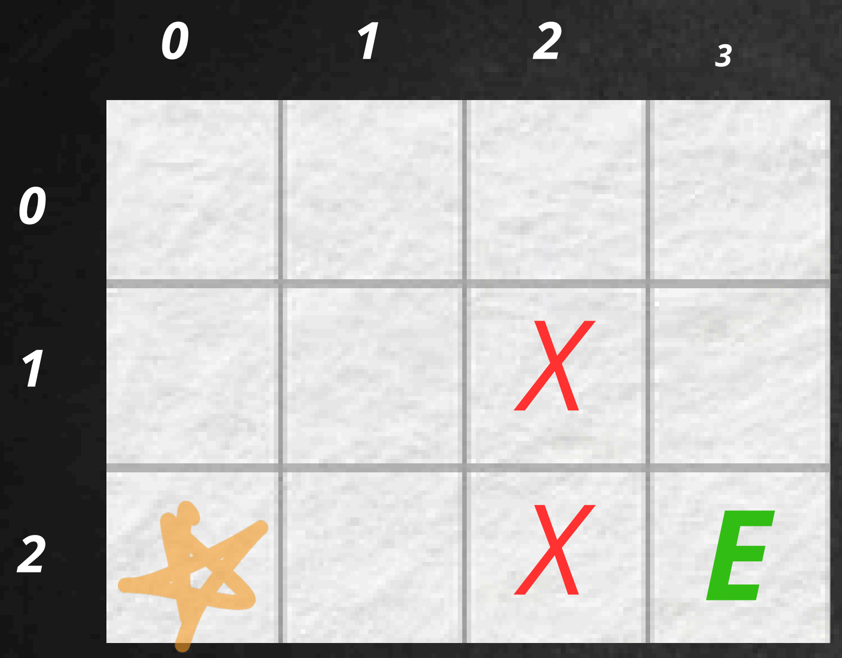
$$P\left(\frac{s_{t+1}, r_{t+1}}{s_t, a_t}\right)$$

-13
-15
-9

APNA COLLEGE

Reinforcement Learning

Grid Example



Policy → random (0, 1, 2, 3)

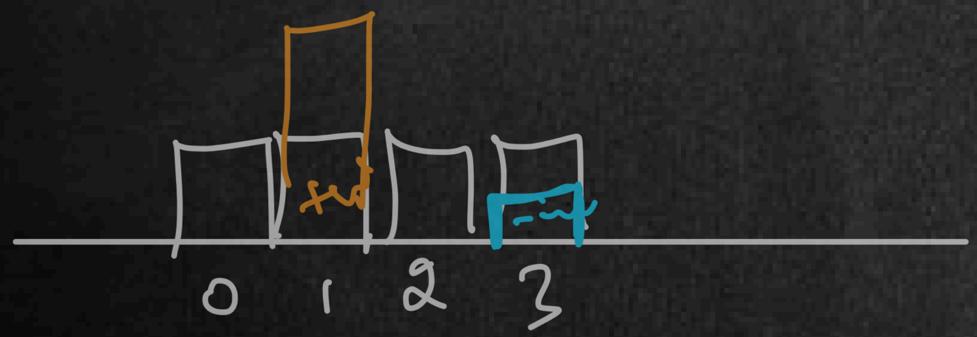
Policy optimization

Policy Gradient Method

State VF
Action VF

series { actions → better results
" " " → worse results

actions → +ve
actions → −ve



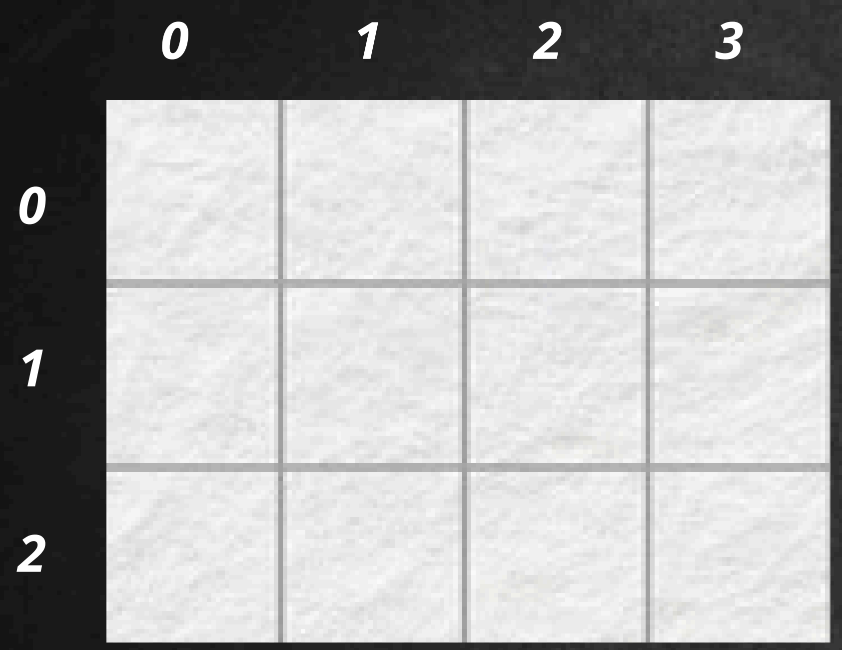
Policy (π)

Reinforcement Learning

Policy $S \rightarrow A$

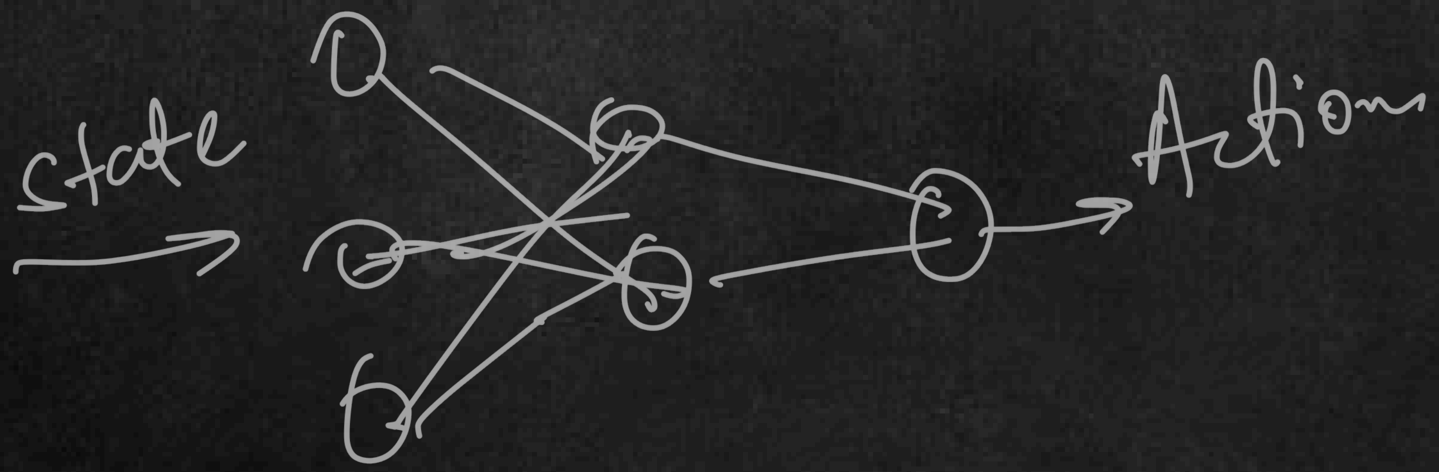
State	Action
(2,0)	A_0
0,0	A_1
1,1	A_2
0,2	A_3

⋮



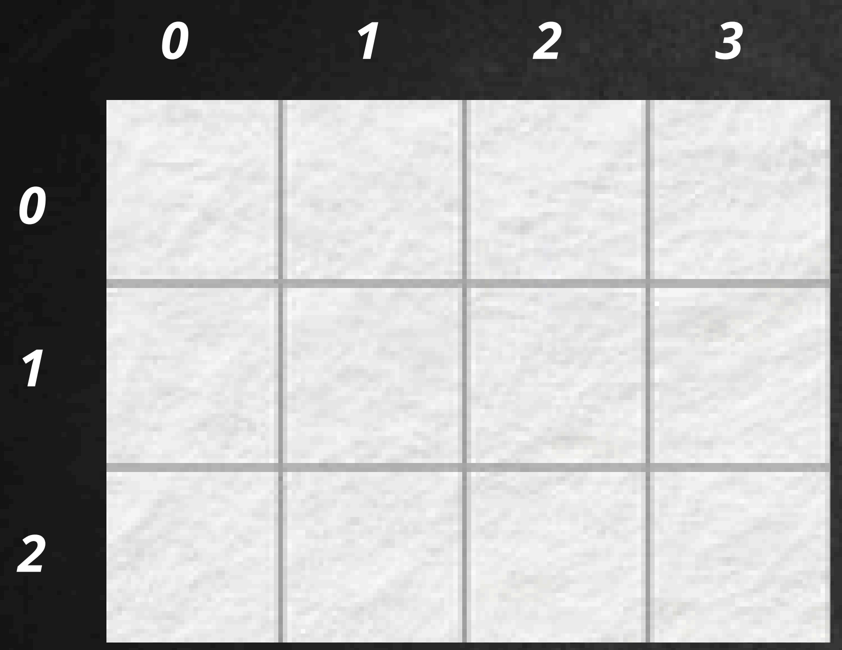
up → 0
down → 1
left → 2
right → 3

neural networks



Reinforcement Learning

Grid Example



model free RL : Action-Value F > State-Value f

Q-function ← $Q_\pi(s, a)$ → estimate the expected returns of S & A for π

$Q_*(s, a)$ → pick the highest estimated return

↓

Greedy

Policy

Q-learning

Deep Q-Networks

Reinforcement Learning

Exploration–Exploitation Tradeoff

explore

new possibilities

apply the learning

best known ans decision

max Action-Value — Action

$\boxed{\epsilon\text{-Greedy strategy}}$ → Q-learning

ϵ (probability to explore)

if(r < ϵ)
    explore
else
    exploit ⇒ optimal Q-val

① ϵ = 0.1 (10% chances to explore)

$0 \le \epsilon \le 1$

②

Action	Q-value
up	5
down	3
left	1
right	10

③ random value (0 to 1) → r

Reinforcement Learning

Exploration–Exploitation Tradeoff

start → explore ↑ exploit ↓
end → " ↓ learning apply ↑

ϵ → gradually decay

↓

probab of explore